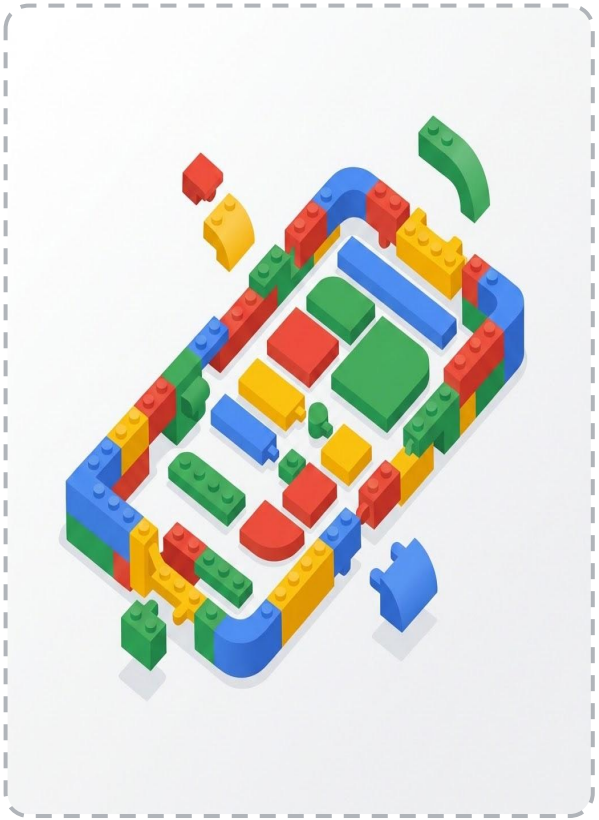
I Inherited an AI-Generated Flutter App.
Here's What It Cost Me

Building apps like LEGO

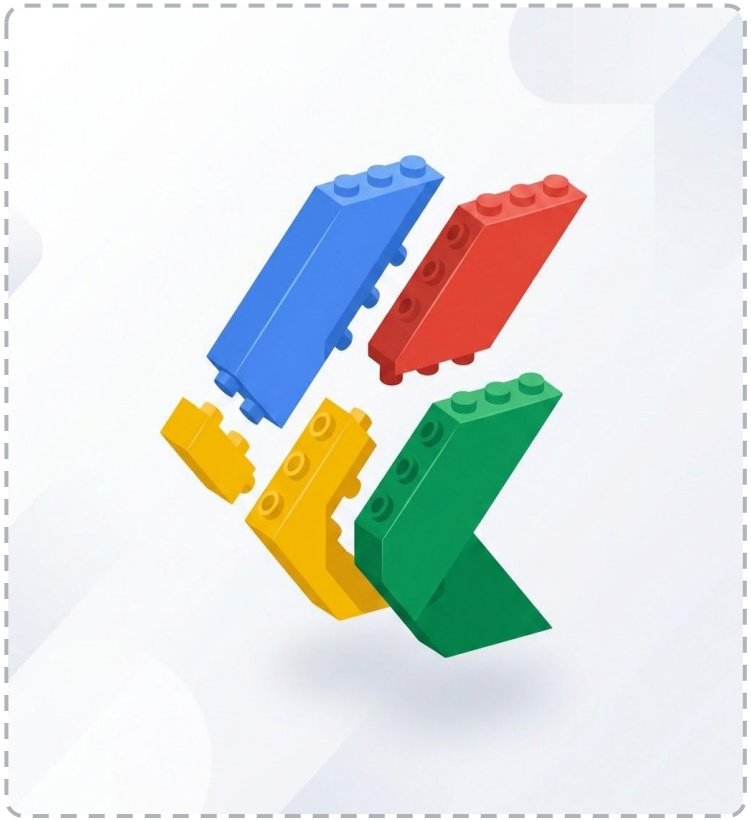
One Core, Many Products



Yashas H Majmudar
Software Engineer, Nest Veterinary

A really good Flutter app already looks like LEGO.

Auth. HTTP clients. Logging. Design system. These should already be bricks you can snap in and out, long before there is ever a second app to prove it.



What counts as a brick

Four modules that should stand on their own inside a single app, before there is ever a reason to share them.

Auth

Login, session, token refresh, isolated from any one screen's flow.

HTTP / Networking Client

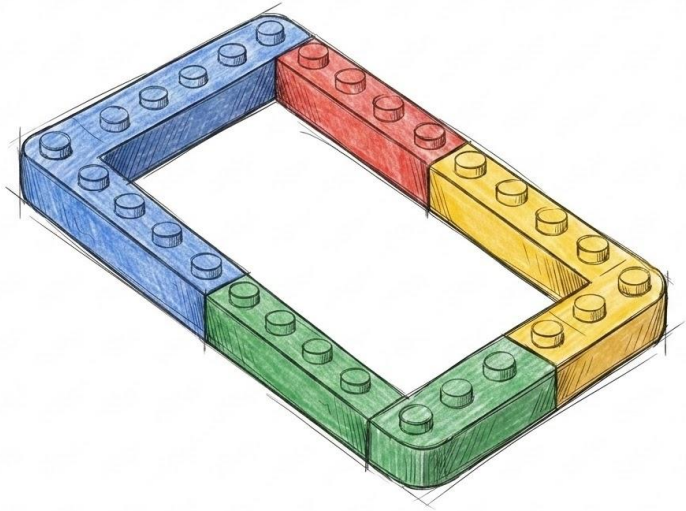
One client, one interceptor chain, one error-handling contract.

Logging

A single logging surface every module reports to, not scattered print statements.

Design System

Tokens and components, not copy-pasted container styling.



What it looks like when you skip it

`app.dart`

~4,000 lines, one file

- Networking calls inlined in widgets
- Magic numbers and hex colors everywhere
- One giant state object for the whole screen
- No shared logging, print statements instead

The four bricks are in there.
They are just fused into
one block instead of four.



One button, before and after

Five jobs fused into one onPressed, versus a widget that only knows contracts.

BEFORE: APP #1, AS INHERITED

```
ElevatedButton(
  onPressed: () async {
    final res = await http.post(
      Uri.parse('https://api.../login'),
      body: {'email': email, 'pw': pw});
    print(res.body);
    setState(() =>
      token = jsonDecode(res.body)['token']);
  },
  style: ElevatedButton.styleFrom(
    backgroundColor: Color(0xFF2E7D6B),
    padding: EdgeInsets.all(13)),
  child: Text('Sign in'),
)
```

AFTER: APP #2, BUILT ON BRICKS

```
AppButton.primary(
  label: 'Sign in',
  onPressed: () => context
      .read<AuthRepository>()
      .signIn(email, pw),
)
```

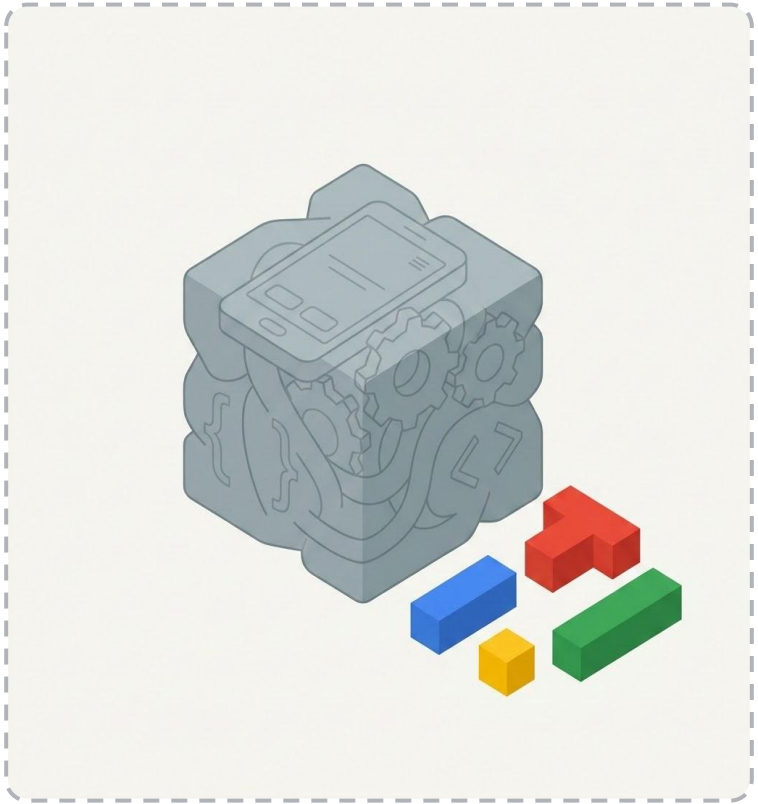
Networking, logging and design tokens now live in packages this widget never imports. There is nowhere left for a magic number to hide.

I inherited an app built mostly by AI.

No design system. No componentization. Magic values scattered through the UI. God files mixing networking, business logic, and UI state. It worked, but nothing in it was a brick, it was one fused block.

What actually worked against me

No auth module. No shared client. No logging surface. No design tokens. Nobody had drawn the lines for the AI to write inside.



The cost, made concrete

When it came time to build app #2, almost nothing carried over. Every one of the four bricks had to be rebuilt from scratch, despite near-identical functionality.



Auth

REBUILT FROM ZERO



HTTP Client

REBUILT FROM ZERO



Logging

REBUILT FROM ZERO



Design System

REBUILT FROM ZERO

~1-2 weeks of otherwise avoidable work, just to give app #2 the infrastructure app #1 already had.

Designing app #2 differently

This time, modularity was a design constraint from day one, not an afterthought.

BEFORE (App #1)

One fused block

Every future app rebuilds auth, clients, logging, and design from scratch.

AFTER (App #2, redesigned)



Four clean module boundaries

A future app #3 needs new UI and business logic only, nothing else.

Bricks are how you steer AI

AI is a very fast new hire. Bricks are the onboarding doc it actually follows.

Contracts

Features depend on AuthRepository, ApiClient and AppLogger interfaces. The easiest code for AI to write is code that uses them.

Guardrails

Lint and CI fail on raw Color(0x…), print(and http imports outside core_network. The model can forget the rules; the build won't.

Context

An AGENTS.md (or CLAUDE.md, Cursor rules) names each brick and says: use these, never reimplement them.

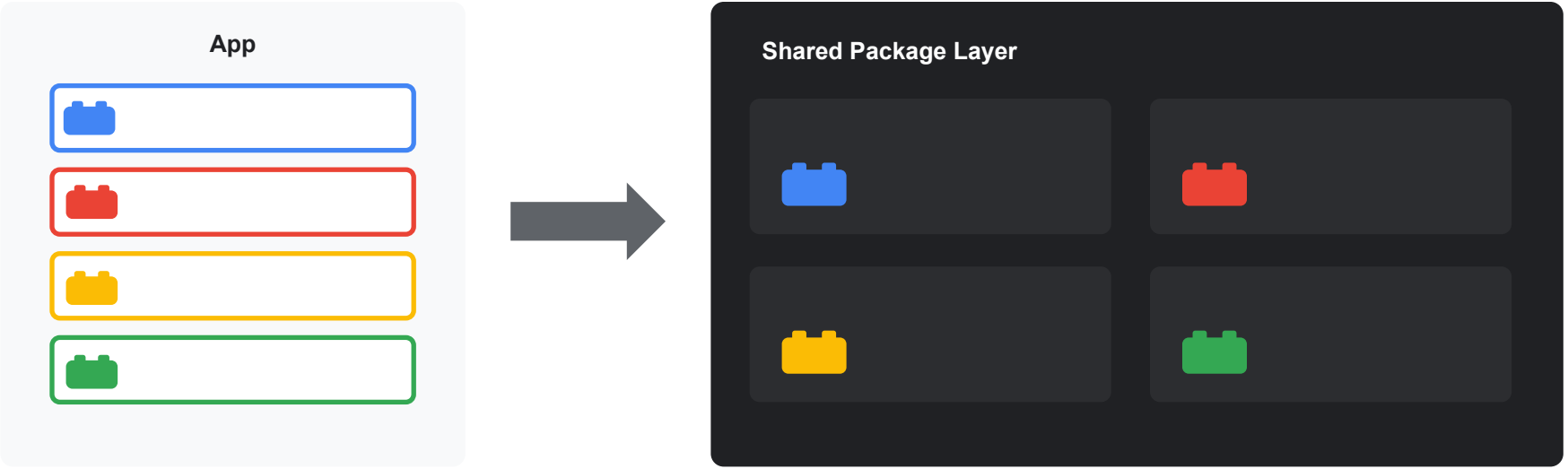
02 PART 2

From One App to Many



The extraction moment

When app #3 arrives, modules built with clear boundaries do not need to be rewritten. They need to be extracted into packages and consumed.



What most teams share vs. keep separate

A practical pattern, not a hard rule.

USUALLY SHARED

- Design system (tokens, components)
- Networking / HTTP client
- Logging
- Auth (often with app-specific wrappers)

USUALLY SEPARATE

- Screens and navigation
- Business logic specific to one app's flow
- Feature flags and app-specific config
- App-specific copy and content

One package, or many?

Genuinely a team-size and preference decision, not a right answer.

ONE CORE PACKAGE

Less overhead, one version to bump, easier for smaller teams to reason about.

```
packages/
  core/
```

Best for: small teams, 2-3 apps, fast iteration.

SPLIT PACKAGES

Independent versioning per module, slower to wire up, more flexible at scale.

```
packages/
  core_auth/
  core_network/
```

Best for: larger teams, 4+ apps, independent release cycles.

Mechanics

root pubspec.yaml: workspace + melos

```
packages/
  core_auth/
  core_network/
  core_logging/
  core_design_system/
apps/
  app_one/
  app_two/
  app_three/
```

Apps outside the workspace pin a version

```
dependencies:
  core_auth: ^2.3.0
  core_network: ^1.8.1
```

Apps inside one workspace share one lockfile and move together.

Per-app pinning needs a private pub server or git tags.

One workspace means one version

Melos 7 runs on Dart pub workspaces: one lockfile resolves every package in the repo.

INSIDE ONE WORKSPACE

Every app resolves the same version of each core package. Apps opt in through flags, DI and config.

```
# root pubspec.yaml
workspace: [packages/*, apps/*]
```

Best for: apps that ship together, from one team.

OUTSIDE THE WORKSPACE

Publish core to a private pub server or tag it in git. Each app pins and upgrades on its own schedule.

```
# app in its own repo
core_auth: ^2.3.0  # private pub
```

Best for: separate teams or repos, independent releases.

When only some apps want the change

Feature flag

New behavior ships dark, apps switch it on when ready.

Use when: the rollout is risky.

Dependency injection

Swap an implementation per app without touching core logic.

Use when: an app needs different behavior.

Config-driven behavior

One codepath, per-app config decides which branch runs.

Use when: same logic, different values.

Versioned package

Apps upgrade on their own schedule, old behavior stays available.

Use when: apps live outside the workspace.

The payoff, with numbers

1-2 days

to stand up a new app's core infra, down from 1-2 weeks

30-45%

of code reused across apps: auth, networking, logging, design system

1 fix

for a shared bug, instead of once per app's own reimplementation

The framework

Modularize as a brick	if more than one part of the app, or a future app, would need it unchanged.
Duplicate on purpose	if the logic is genuinely app-specific and sharing it would force a false abstraction.
Extract to a package	once a second consumer actually exists, not before. Speculative packages rot.
Revisit the boundary	the moment a core change starts needing app-specific exceptions.

A good Flutter app was already LEGO.

The second app just makes you pay for skipping it, or rewards you for not.

These boundaries are not a reaction to AI. They are what makes AI-generated code safe to ship.



One brick at a time



A single app should already be modular.



The real cost shows up on app number two.



Extract to a package once a second consumer exists.

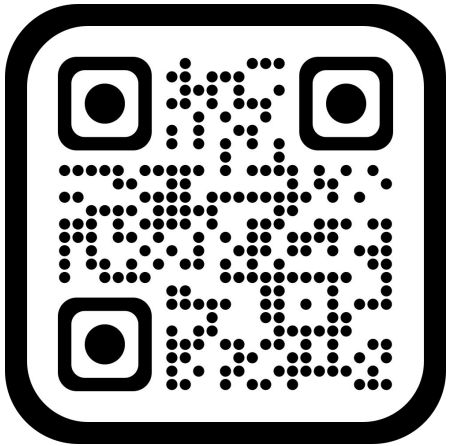


Most teams share core infra, keep screens and logic separate.



Bricks give AI contracts, guardrails and context to work within.

Thank you.



Yashas H Majmudar
Software Engineer, Nest Veterinary

Also working on production AI integrations in Flutter, and the architectural patterns that keep AI-assisted development scalable. Come find me after.